

Codebase Change Summary — Week 6

Data-Flow Testing and Coverage Concepts

SE 413/513 — Software Testing

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 5, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 6 and, more importantly, explains *why* each change exists — which concept from the Week 6 reading, *Data-Flow Testing and Coverage Concepts*, it demonstrates. Read this alongside the reading and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–5 was removed or rewritten. Everything below is *additive*: the running examples you already saw are all still there, unchanged, including their two intentional `xfail` results and one intentional `FAIL`.

2 What Changed At a Glance

File	Status	Purpose
<i>The main suite</i>		
app/billing/charges.py	New	The reading's worked example, <code>sum_positive()</code> , verbatim with its S1–S5 labels: sums the charges on a statement, skipping credits.
tests/billing/test_charges.py	New	Four tests whose ids name the coverage criterion and the DU pairs each one covers, including the loop-carried pair.
<i>Deliberately buggy illustrations (not in the default run)</i>		
demos/data_flow/accumulator_bug.py	New	<code>sum_positive()</code> with the loop-carried pair broken. 100% branch coverage; one All-Uses test fails.
demos/data_flow/receipt.py	New	A correct function where a one-test All-Defs set leaves a branch untested (75%).
demos/data_flow/anomalies.py	New	A use-before-def and a dead definition that tests pass over and <code>pylint</code> finds statically.
<i>Tooling and documentation</i>		
requirements.txt	Modified	Adds <code>pylint</code> .
docs/DEMO_GUIDE_week06_data_flow.md	New	Section-by-section lecture companion for the reading.
README.md (suite and repository)	Modified	Updated layout, <code>pylint</code> command, expected output, and the Weekly Updates index.

Tip

Install the new dependency before running anything this week:

```
cd demo-testing-suite
pip install -r requirements.txt
```

3 The Worked Example: `sum_positive()`

Locations: `app/billing/charges.py`, `tests/billing/test_charges.py`

```
def sum_positive(numbers):
    total = 0                # S1
    for n in numbers:        # S2
        if n > 0:            # S3
            total = total + n # S4
    return total             # S5
```

The module docstring repeats the reading's def/use table. The test file's docstring repeats its seven DU pairs, with a “covered by” column. Each test id says which criterion the test belongs to:

Test id	Input	Result	New DU pairs (reading's numbering)
all-defs:credit-only	[-1]	0	#2, #6
all-defs:one-charge	[3]	3	#1, #4, #5, #7
all-uses:two-charges	[3, 5]	8	#3 (loop-carried)
du-path:loop-carried-via-credit	[3, -2, 5]	8	none new: #3 along a second def-clear path

```
pytest tests/billing/test_charges.py -v \
--cov=app.billing.charges --cov-branch --cov-report=term-missing
```

The first two tests already give 100% branch coverage of this function. The third adds no statement and no branch — it adds a *pair*. The fourth reaches pair #3 through the other back edge (S3's false outcome, a skipped credit between two charges). It covers no new DU pair, so All-Uses does not require it; it is the kind of extra path All-DU-Paths asks for.

4 Why the Loop-Carried Pair Matters

Locations: demos/data_flow/accumulator_bug.py, demos/data_flow/test_accumulator_bug.py

The same function, with S4 broken:

```
total = 0 + n      # S4 KNOWN BUG -- should be total + n
```

Each charge now overwrites (*kills*) the running total instead of adding to it, and pair #3 no longer exists. The demo's tests are the All-Defs set plus one All-Uses test:

```
pytest demos/data_flow/test_accumulator_bug.py -v \
--cov=accumulator_bug --cov-branch --cov-report=term-missing
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
demos/data_flow/accumulator_bug.py	6	0	4	0	100%	

The two All-Defs tests pass, with **100% statement and branch coverage**. Only `test_all_uses_two_charges_exposes_the_bug` fails, with `assert 5 == 8`: the function returns the last charge, not the sum.

Common Pitfall

Branch coverage confirms every decision ran both ways. It says nothing about whether an accumulated value was ever checked after more than one update. Whenever a variable is both defined and used inside a loop, ask whether the definition also reaches a use on a *later* iteration.

5 All-Defs Does Not Subsume Branch Coverage

Locations: demos/data_flow/receipt.py, demos/data_flow/test_receipt.py

The reading states this but its own example happens not to show it: in `sum_positive()`, the All-Defs set also covers both branches. `total_with_tax(amount, show_breakdown)` does show it. A single call with `show_breakdown=False` gives every definition one use, which is All-Defs:

```
pytest demos/data_flow/test_receipt.py \
    --cov=receipt --cov-branch --cov-report=term-missing
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing

demos/data_flow/receipt.py	6	1	2	1	75%	33

Line 33, the `print()` on the `True` branch, never ran. Uncomment `test_total_with_breakdown` at the bottom of the test file: it covers the remaining DU pairs (All-Uses) and the report reaches 100%.

6 Anomalies That Static Analysis Finds

Locations: `demos/data_flow/anomalies.py`, `demos/data_flow/test_anomalies.py`

Two of the reading's defect patterns, shipped on purpose:

- **Use before def:** `describe_balance()` never assigns `status` when the balance is exactly 0.
- **Dead definition:** `apply_payment()` computes `remaining` and then returns the wrong variable.

The three tests pass, and every statement runs. `pylint` needs no tests at all:

```
pylint demos/data_flow/anomalies.py --disable=all \
    --enable=possibly-used-before-assignment,unused-variable
```

```
anomalies.py:36:25: E0606: Possibly using variable 'status' before assignment
anomalies.py:40:4: W0612: Unused variable 'remaining'
```

This is the reading's "Data-Flow Testing in Practice" section: data-flow *coverage* tools are rare, but def-use *analysis* runs inside everyday linters, compilers, and IDEs.

Tip

In Python, a use before def crashes with `UnboundLocalError` (uncomment `test_zero_balance` to see it). In some other languages the same mistake silently returns garbage or a stale value, which is why the reading treats it as a data-flow defect, not just a crash.

7 Updated: Expected Test Results

From the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
pytest -v
```

you should now see **69 passed, 2 xfailed, 1 failed**: four more passing tests than Week 5's baseline, and the same two xfail results and one FAIL as before, unrelated to this week's additions. `demos/data_flow/` is not part of that run (`pytest.ini` sets `testpaths = tests`); run it by path. Its one failing test is intentional.

8 How This Connects to the Reading

Key Idea

Week 5's coverage report told you *which code ran*. This week's files ask *whether the right values got where they needed to go*. The same function, `sum_positive()`, appears twice: once correct, with tests chosen pair by pair, and once broken, with a test set that satisfies All-Defs and 100% branch coverage and still misses the bug. The difference between them is one DU pair.

9 Suggested Next Steps

1. Before opening `tests/billing/test_charges.py`, list the seven DU pairs of `sum_positive()` yourself and choose a minimal All-Uses test set. Then compare with the test ids.
2. Fix `demos/data_flow/accumulator_bug.py` and confirm its failing test passes, with the coverage report unchanged at 100%.
3. Fix both functions in `demos/data_flow/anomalies.py` so that `pylint` reports nothing. Add the boundary test that should have caught the first one.
4. Work the reading's `first_negative()` exercises on paper, then write the function and a parametrized test whose ids name the DU pairs, in the style of `test_charges.py`.
5. The Week 5 coverage gap in `app/orders/order_service.py` is still open. If you have not closed it yet, do that too.